

# SIMATS ENGINEERING ASSESSMENT TOOL

COURSE NAME: Theory of Computation COURSE CODE: CSA13

COURSE OUTCOME COVERED

**CO2: Deign Finite Automata DFA, NFA,  $\epsilon$ -NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)**

*Assessment Tool Weightage: 20%*

QUESTIONS: 5 Total Marks:20 Duration : 1 Hour

## Scenario Based

*Code of Conduct:*

*I **Kadava Yogi (192372053)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

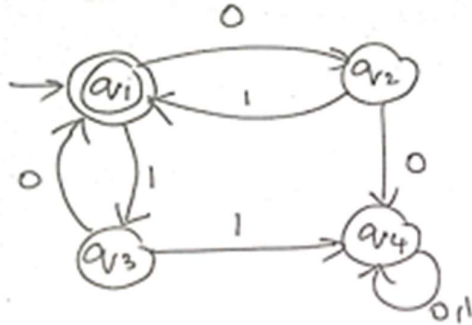
*I understand that any violation of academic integrity rules will result in disciplinary*

*action. Signature of the student:*

# SIMATS ENGINEERING ASSESSMENT TOOL

1. Given a DFA used for validating a specific PIN pattern bbbc, bca, c, ca, caaaaa minimize the DFA and explain how state reduction improves performance and efficiency in embedded systems

2. Demonstrate how a given finite automaton can be converted into a regular expression



3. Consider the following grammar

$S \rightarrow aSc | T | U | e$

$T \rightarrow bTc | e$

$U \rightarrow aUb | e$

A) What is the language defined by the grammar?

B) Give the derivation tree for the word aaabbc

C Grammar is Ambiguous if there exists two distinct derivations for a word. Is the above grammar ambiguous? Justify.

4. I) Show that if  $L_1$  and  $L_2$  are regular then  $(L_1 + L_2)$  also regular

ii) Is the language  $\{a^p \mid p \text{ is prime}\}$  regular? Justify

# SIMATS ENGINEERING ASSESSMENT TOOL

:

**RUBRICS FOR CALCULATION**

| Criteria                 | Weightage | Level 4 – Exemplary (5 Marks)                                | Level 3 – Proficient (4 Marks)         | Level 2 – Developing (2–3 Marks)             | Level 1 – Beginning (0–1 Mark)  |
|--------------------------|-----------|--------------------------------------------------------------|----------------------------------------|----------------------------------------------|---------------------------------|
| Conceptual Understanding | 30        | Demonstrates clear and complete understanding of the concept | Good understanding with minor gaps     | Basic understanding with some misconceptions | Poor or incorrect understanding |
| Accuracy of Answer       | 25        | Completely correct answer with no errors                     | Mostly correct with minor errors       | Partially correct with noticeable errors     | Incorrect or irrelevant answer  |
| Application of Knowledge | 20        | Correctly applies concepts to solve the question/problem     | Applies concepts with minor mistakes   | Limited or partially correct application     | No or incorrect application     |
| Completeness of Response | 15        | Covers all required parts of the question                    | Covers most parts with minor omissions | Covers only some parts                       | Incomplete or missing response  |
| Clarity & Presentation   | 10        | Clear, concise, and well organized answer                    | Understandable with minor issues       | Somewhat unclear or poorly structured        | Difficult to understand         |

## 1) Minimize the DFA for PIN patterns

**PINs:** bbbc, bca, c, ca, caaaaaa

### Minimized DFA (States)

| State | Meaning    | Input a | Input b | Input c | Final             |
|-------|------------|---------|---------|---------|-------------------|
| q0    | Start      | q4      | q1      | q3      | No                |
| q1    | Seen b     | Dead    | q2      | q5      | No                |
| q2    | Seen bb    | Dead    | q6      | Dead    | No                |
| q3    | Seen c     | q4      | Dead    | Dead    | Yes (c)           |
| q4    | Seen ca... | q4      | Dead    | Dead    | Yes (ca, caaaaaa) |
| q5    | Seen bc    | q4      | Dead    | Dead    | Yes (bca)         |
| q6    | Seen bbb   | Dead    | Dead    | q7      | No                |
| q7    | Seen bbbc  | Dead    | Dead    | Dead    | Yes (bbbc)        |
| Dead  | Trap       | Dead    | Dead    | Dead    | No                |

### State Reduction Benefits

- Removes equivalent/redundant states.
- Reduces memory required to store transition tables.
- Faster state transitions, improving execution speed.
- Saves power and processor time in embedded systems.
- Simplifies implementation and debugging.

---

## 2) Convert the given FA to Regular Expression

From the diagram:

- Start state: q1
- Final state: q1
- $q1 \leftrightarrow q2$  on **0,1**
- $q1 \leftrightarrow q3$  on **0,1**
- $q2 \rightarrow q4$  on **0**
- $q3 \rightarrow q4$  on **1**
- q4 has loop on **0,1**

The accepted strings are those that start and end at q1.

### Regular Expression

$$((0 + 1)(0 + 1))^*$$

or equivalently,

$$(00 + 01 + 10 + 11)^*$$

---

### 3) Grammar

Given

$S \rightarrow aSc \mid T \mid U \mid \varepsilon$

$T \rightarrow bTc \mid \varepsilon$

$U \rightarrow aUb \mid \varepsilon$

#### (A) Language Defined

The grammar generates

- Equal number of **a's** and **c's**
- Equal number of **b's** and **c's**
- Equal number of **a's** and **b's**

Hence,

$$L = \{a^n c^n\} \cup \{b^n c^n\} \cup \{a^n b^n\}$$

including  $\varepsilon$ .

---

#### (B) Derivation Tree for aaabbcc

S

|

a

|

S

|

a

|

S

|

a

|

U

|

U

|

$\varepsilon$

(Equivalent leftmost derivation)

S

$\Rightarrow aSc$

$\Rightarrow aaSc$

$\Rightarrow aaaUcc$

$\Rightarrow$  aaaUbcc

$\Rightarrow$  aaabbcc

---

**(C) Is the grammar ambiguous?**

**Yes.**

A grammar is ambiguous if one string has two different parse trees.

Since  $\epsilon$  can be generated from

$S \rightarrow \epsilon$

$S \rightarrow T \rightarrow \epsilon$

$S \rightarrow U \rightarrow \epsilon$

there are multiple derivations for  $\epsilon$ .

Hence, **the grammar is ambiguous.**

---

**4) Show that if  $L_1$  and  $L_2$  are regular then  $(L_1 + L_2)$  is regular**

**Proof**

Let

- DFA  $M_1$  recognize  $L_1$
- DFA  $M_2$  recognize  $L_2$

Construct an NFA with a new start state having  $\epsilon$ -transitions to the start states of  $M_1$  and  $M_2$ .

This NFA accepts

$$L_1 \cup L_2$$

Since every NFA has an equivalent DFA, the union language is also regular.

Therefore,

$$L_1 + L_2$$

is **regular**.

---

**4(ii) Is  $L = \{a^p \mid p \text{ is prime}\}$  regular?**

**Answer**

**No.**

**Justification (Pumping Lemma)**

Assume the language is regular.

Let pumping length be  $m$ .

Choose

$$w = a^q$$

where  $q$  is a prime number greater than  $m$ .

By the pumping lemma,

$$w = xyz$$

with

- $|xy| \leq m$
- $|y| > 0$

Pump  $y$  twice:

$$xy^2z$$

Its length becomes

$$q + |y|$$

or by choosing an appropriate pumping multiple,

$$q + (k - 1) |y| = q \left( 1 + \frac{|y|}{q} \right)$$

which can be made **composite**, so the pumped string is **not** in the language.

This contradicts the pumping lemma.

Hence,

$$L = \{a^p \mid p \text{ is prime}\}$$

is **not regular**.